ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Expresiones y Estructuras de Control

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase Activa: Módulo 1

Ejercicio 1.1

Cree un programa que solicite por pantalla una temperatura en grados Fahrenheit y arroje como resultado su transformación a Grados Celsius.

$$^{\circ}C = \frac{5}{9} \times (^{\circ}F - 32)$$

Aprendemos: Entrada y Salida, Tipos de Datos, Conversiones, Operadores Aritméticos.

Clase Activa: Módulo 1

Ejercicio 1.2

Cree un programa que solicite por pantalla una temperatura en grados Celsius y pregunte si quiere como resultado su transformación a Grados Fahrenheit o Kelvin.

$$^{\circ}K = {}^{\circ}C + 273{,}15$$

Aprendemos: control condicional, if, u opciones, switch.

Clase Activa: Módulo 1

Ejercicio 1.3

Cree un programa que pregunte por pantalla un tipo de temperatura y un valor, y luego pregunte en que otra unidad quiere transformarlo. (por ejemplo, inicia como Kelvin, y se entrega Celsius o Fahrenheit).

$$^{\circ}F = \frac{9}{5} \times (^{\circ}K - 273{,}15) + 32$$

Aprendemos: control condicional, if, u opciones, switch.

Clase Activa: Módulo 1

Ejercicio Desafío

Cree un programa que pregunte un día de la semana y luego una cantidad X de días. El programa debe retornar el día de la semana que será en X días.

Aprendemos: Aritmética Modular, control de condición u opción.

Clase Activa: Módulo 2

Ejercicio 2.1

Cree un programa que solicite un número positivo entero y verifique si éste es primo o compuesto. No puede utilizar **break**.

Aprendemos: Ciclos, Aritmética Modular.

Clase Activa: Módulo 2

Ejercicio 2.2

Cree un programa que solicite un número positivo entero y verifique si éste es primo o compuesto. Debe utilizar **break**.

Aprendemos: Ciclos, Aritmética Modular. Comandos especiales

Clase Activa: Módulo 3

Ejercicio 3.1

Cree un programa que solicite un número entero largo, y luego calcule cuántas veces aparece cada dígito en él. Si posee todos los dígitos pares se debe decir por pantalla que es un parista, y si posee todos los números impares, debe decir que es un imparista.

Aprendemos: Ciclos, Corto Circuito

Clase Activa: Módulo 3

Ejercicio Desafío

Cree un programa que solicite una palabra, y calcule cuántas veces aparece cada letra del abecedario en ella. Luego, si posee todas las vocales debe decir que es una palabra **vocal total**.

Aprendemos: Ciclos, Corto Circuito

Información de Vital Importancia para la Clase Activa[1].

[1] Depende exclusivamente de uds. leerla y comprenderla.

Expresiones en Asignaciones

- **Valor de una expresión** determinado por **reglas de orden de evaluación de operadores y operandos**:
  - Asociatividad.
  - Precedencia.
- Si **orden es ambiguo** puede conducir a **diferentes resultados**, **según la implementación** del traductor de lenguaje.
- Tip: Usar paréntesis ante la duda.

Expresiones Aritméticas

Conjunto de **operadores**, **operandos** (argumentos), **paréntesis** y **llamadas a funciones**. El **orden de evaluación** (precedencia, asociatividad y paréntesis) determina su **valor**.

- **Aridad de operadores:**
  - *Unario (unitario):* Un sólo operando.
  - *Binario:* Dos operandos.
  - *Ternario:* Tres operandos.

Operadores Unarios

Operador	C	Python
Negación lógica	!	not
Dirección	&	*No existe*
Derreferencia	*	*No existe*
Incremento	++	*No existe*
Decremento	- -	*No existe*
Signo más	+	+
Negativo	-	-
Casting	(tipo)	*No existe*

Los operadores unarios agrupan de derecha a izquierda.
E.g. $-+x$ equivalente a $-(+x)$.

Operadores Binarios

Operador	C	Python
Suma	+	+
Resta	-	-
Multiplicación	*	*
División	/	/
División entera	/	//
Módulo	%	%
Exponenciación	*No existe*	**

División entera en C según tipo de dato.

Si tipo entero para numerador y denominador: división entera.

Orden de Evaluación de Operadores

- **Precedencia:** Orden de evaluación de operadores adyacentes, estableciendo prioridades.
- **Asociatividad:** A igual precedencia, define orden de evaluación.
- **Paréntesis:** Altera y fuerza orden de evaluación.

Precedencia	Descripción	Operador		Asociatividad
		C	Python	
alta	exponente		**	←
	unarios (postfijos)	++ −−		→
	unarios (prefijos)	++ −− ~ ! * & + -	~ + -	←
	casting de tipo	(type)		←
	multiplicativos	* / %	* / // %	→
	aditivos	+ −	+ −	→
baja	NOT lógico		**not**	←

Ojo con conversiones!

- Expresión puede tener muchos tipos distintos.
- Entonces, el lenguaje hace conversión implícita (coerción), para usar el operador adecuado.
- Mayoría de lenguajes usa conversión por expansión.
- **Pro:** Da más flexibilidad a uso de operadores.
- **Contra:** Reduce la capacidad de detectar algunos errores e introduce código adicional.
- Control **implícito** de conversión mediante *coerción*:

```
float    fVal;
double   dVal;
int      iVal;
unsigned long ulVal;

dVal = iVal * ulVal; //iVal es convertido a unsigned long.
// El resultado es convertido a double

dVal = ulVal + fVal; //ulVal convertido a float.
//El resultado es convertido a double
```

Sentencias de asignación

- Permiten cambiar dinámicamente el valor ligado a una variable.
- C permite **incrustar una asignación en una expresión** (actúa como cualquier operador binario).
- Existen diversos operadores que integran operadores binarios con asignación. Ejemplo:

```
a *= 5;
//Equivalente a:
a = a * 5;
// Todos siguen misma estructura.
```

Sentencias de Asignación

Operador	**C**	**Python**
Asignación	$=$	$=$
Suma y asignación	$+=$	$+=$
Resta y asignación	$-=$	$-=$
Multiplicación y asignación	$*=$	$*=$
División y asignación	$/=$	$/=$
Módulo y asignación	$\%=$	$\%=$
Incremento	$++$	
Decremento	$--$	
División entera y asignación		$//=$
Exponente y asignación		$**=$

Sentencias de Asignación

Operadores Compuestos y Unarios de Asignación

Unarios de Asignación - C:

```
//pre-incremental
sum = ++contador;

//equivale a:

contador = contador + 1;
sum = contador;
```

```
//post-incremental
sum = contador++;

//equivale a:
aux = contador;
contador = contador + 1;
sum = aux;
```

Operadores Compuestos - C/Python:

```
sum += a;

//equivale a:
sum = sum + a;
```

Sentencias de Asignación

Asignación Múltiple

Asignación múltiple simple.

C

```
int a, b, c, d;
a = b = c = d = 5;
```

Python

```
a = b = c = d = 5
```

Asignación múltiple compuesta.

C

```
int a = 10, b, c = 15;
a += b = c -= 5;
```

Python (lista)

```
x, y, z = 20, 30, 40
```

Sentencias de Asignación

Operador Ternario

- **C: Único operador ternario.**

```
retorno = op1 ? op2 : op3;
```

Si op1 es verdadero, asigna op2; sino, asigna op3.

```
cuenta = (a > 0) ? 0 : 1;

//Equivale a:
if (a > 0)
        cuenta = 0;
else cuenta = 1;
```

- **Python:**
  retorno = op2 **if** op1 **else** op3

Asignación en expresiones

- C considera la asignación como un operador más.
- Permite incrustar asignación en cualquier expresión.
- **C** Permitido.

```
while ( (c = getchar()) != EOF) {}
```

- **Python:** Sintaxis no lo permite.

```
if (d = 5 + c) > 6:
```

Error: Invalid syntax.

- Pro: Programación más compacta.
- Contra: Fuente de error en la programación. Es fácil equivocarse confundiendo == e =.

Expresiones Relacionales y Lógicas

- **Expresión Relacional:**
  - Usan operador relacional (binario) para comparar los valores de dos operandos.
  - Cada operando puede ser una **expresión aritmética**.
  - El **valor resultante** de la expresión es **booleano**.
  - Incluye: $==$, $!=$, $<$, $<=$, $>$, $>=$
- **Expresión Booleana:**
  - Consiste en variables, constantes y operadores booleanos.
  - Se puede combinar con expresiones relacionales.
  - Incluye: *AND*, *OR*, *NOT* y, a veces, *XOR*.
- **El único valor falso en C es el 0 (cero).**

Expresiones Relacionales y Lógicas

Operadores Relacionales		
Operador	**C**	**Python**
Igual	==	==
Distinto	! =	! = <>
Mayor	>	>
Menor	<	<
Mayor o igual	>=	>=
Menor o igual	<=	<=
Operadores Booleanos		
Operador	**C**	**Python**
NOT	!	**not**
AND	&&	**and**
OR	||	**or**

Estructuras de control

- C/Python la ejecución es típicamente secuencial.
- Dos tipos básicos de estructuras control:
  - **Condicional:** múltiples alternativas de ejecución, controladas por una condición.
  - **Iterativa:** Ejecución repetitiva de un grupo de sentencias, también controladas por una condición.
- Se requiere mecanismo de agrupación de sentencias sobre las cuales se ejerce el control: **composición de sentencias**.
- En **C** se usa { }
- **Python** compone por indentación.

Estructuras de control - Sentencias de selección

if - else if - else

C

```
if(condición_1) {
    //....
} else if(condición_k) {
    //....
} else {
    //....
}
```

Python

```
if condición_1:
    #....
elif condición_k:
    #....
else:
    #....
```

Estructuras de control - Sentencias de selección

switch

C

```
switch(x){ //Debe ser una variable
case x1:
        //....
        break;
case x2:
        //....
        break;
//....
case xN:
        //....
        break;
default: //opcional
        //....
}
```

No existe en Python.

Estructuras de control - Sentencias de iteración

Ciclos controlados por condicion: *while, do-while*

C

```
while(condición) {
    //...
}

do {
    //...
} while(condición);
```

Python:

```
while condición:
    #...
```

Estructuras de control - Sentencias de iteración

Ciclos controlados por contador: *for*

C

```c
//En general:
for(sentencia_inicial;condición;sentencia_fin_ciclo) {
    //...
}
```

C

```c
//Uso corriente
for(i=0; i<N; i++) {
    //...
}

//Equivalente a:
i=0;
while(i<N) {
    //...
    i++;
}
```

Python:

```python
//En general:
for var_iteracion in secuencia:
    #instrucciones ...
```

```python
for letra in 'Py':     # Ejemplo 1
   print 'Letra actual:', letra

frutas = ['platano', 'manzana', 'mango']
for fruta in frutas:       # Ejemplo 2
   print 'Fruta actual:', fruta
```

```
Letra actual: P
Letra actual: y
Fruta actual: platano
Fruta actual: manzana
Fruta actual: mango
```

Corto Circuito o Término anticipado de evaluación

- **Corto circuito:** Evaluación de una expresión termina antes.
- **C** definen corto circuito para && y ||.
- Lenguajes como **Perl y Ruby** evalúan todos sus operandos en una expresión lógica.

```
//Si primera condición se cumple, la expresion es true
if((a >= 0) || (b < 10))

//Si primera condición no se cumple, la expresion es false
while ( (c = getchar()) != EOF && c != '\n' )
```

- Puede generar inconvenientes en expresiones con efectos colaterales.

Saltos incondicionales

- **C/Python:**
  - **break:** Permite salir del ciclo más cercano.
  - **continue:** Permite pasar al ciclo siguiente.
  - Se recomienda usarlos con precaución y critero.

Saltos incondicionales - Ejemplo

```
int main() {
  int n;
  do {
    printf("Ingrese numero: ");
    scanf("%d", &n);
    if (n < 0)
      break;
    if (n >10) {
      printf("Saltarse el valor.\n");
      continue;
    }
    printf("El numero es: %d\n",  n);
  } while (n != 0);

  return 0;
}
```